

PPL Assignment 4

Stav Mordekhai (213250400), Tamar Shefi Simchon (212550966)

Question 1.1: Lazy Lists

a. Equivalence criterion for two lazy lists:

Two lazy lists L_1 and L_2 are equivalent if they generate the exact same finite or infinite sequence of elements. Formally, for any non-negative integer n , retrieving the first n elements using the `take` procedure yields the exact same standard list for both lazy lists.

$$\forall n \geq 0, \text{take}(L_1, n) = \text{take}(L_2, n)$$

b. Showing even-square-1 and even-square-2 are equivalent:

Both lazy lists operate on the infinite sequence generated by `(integers-from 0)`, which is $(0, 1, 2, 3, 4, 5, \dots)$.

- **even-square-1** applies `lz1-map` first, squaring every integer to produce $(0, 1, 4, 9, 16, 25, \dots)$. Then, it applies `lz1-filter` to keep only the even numbers. The resulting sequence is $(0, 4, 16, 36, \dots)$.
- **even-square-2** applies `lz1-filter` first, keeping only the even numbers from the base sequence to produce $(0, 2, 4, 6, \dots)$. Then, it applies `lz1-map` to square those remaining numbers. The resulting sequence is $(0^2, 2^2, 4^2, 6^2, \dots)$, which equals $(0, 4, 16, 36, \dots)$.

Because both sequences generate the exact same elements in the exact same order, $\text{take}(\text{even-square-1}, n) = \text{take}(\text{even-square-2}, n)$ for any n . Thus, they are equivalent.

Question 2: CPS

a. Equivalence criterion for a procedure and its CPS version:

Let f be a standard procedure of type $A \rightarrow B \cup \{\text{'fail'}\}$. Let $f\$$ be its Success-Fail-Continuations version of type $A \times (B \rightarrow C) \times (\emptyset \rightarrow D) \rightarrow C \cup D$.

f and $f\$$ are equivalent if for all valid inputs x :

- If $f(x)$ successfully returns a value v (where $v \neq \text{'fail'}$), then $f\$(x, \text{success}, \text{fail})$ invokes the success continuation with that value: `(success v)`.
- If $f(x)$ evaluates to `'fail'`, then $f\$(x, \text{success}, \text{fail})$ invokes the fail continuation: `(fail)`.

d. Showing get-value and get-value\$ are equivalent:

We can prove this by examining the three branches of both `cond` statements for a given `key` and `assoc-list`:

- **Base Case (Empty List):** If `assoc-list` is empty, `get-value` returns `'fail'`. Under the same condition, `get-value$` evaluates to `(fail)`, correctly invoking the fail continuation.
- **Match Case:** If the key matches the head of the list (`eq? key (caar assoc-list)`), `get-value` returns the value `(cdar assoc-list)`. Under the same condition, `get-value$` applies the success continuation to the value: `(success (cdar assoc-list))`.
- **Recursive Case:** If the key is not at the head, `get-value` recurses on `(cdr assoc-list)`. Similarly, `get-value$` recurses on `(cdr assoc-list)` while passing along the exact same success and fail continuations.

By induction, since the base cases satisfy the equivalence criterion and the recursive step preserves it, `get-value` and `get-value$` are equivalent.

Question 3.1: Unification

1. **unify** [$t(s(s), G, s, p, t(K), s), t(s(G), G, s, p, t(K), U)$]
 - **Step 1:** Unify the outer functors: $t = t$. Proceed to the arguments sequentially.
 - **Step 2:** Unify $s(s) = s(G) \Rightarrow s = G \Rightarrow G = s$. Apply the substitution $\{G/s\}$ to the rest of the terms.
 - **Step 3:** Unify $G = G \Rightarrow s = s$. Success.
 - **Step 4:** Unify $s = s$. Success.
 - **Step 5:** Unify $p = p$. Success.
 - **Step 6:** Unify $t(K) = t(K) \Rightarrow K = K$. Success.
 - **Step 7:** Unify $s = U \Rightarrow U = s$. Apply the substitution $\{U/s\}$.
 - **Result: Success.** The substitution composition is $\{G/s, U/s\}$.
2. **unify**[$g(1, M, g, G, U, g, v(M)), g(1, v(U), g, v(M), v(G), g, v(M))$]
 - **Step 1:** Unify the outer functors: $g = g$. Proceed to arguments.
 - **Step 2:** Unify $1 = 1$. Success.
 - **Step 3:** Unify $M = v(U)$. Apply substitution $\{M/v(U)\}$.
 - **Step 4:** Unify $g = g$. Success.
 - **Step 5:** Unify $G = v(M)$. Applying the previous substitution, we get $G = v(v(U))$. Apply $\{G/v(v(U))\}$.
 - **Step 6:** Unify $U = v(G)$. Applying the previous substitution, we get $U = v(v(v(U)))$.
 - **Result: Failure** due to the **Occurs Check**. The variable U occurs within the composite term $v(v(v(U)))$, so it cannot be unified.
3. **unify** [$m(M, N), n(M, N)$]
 - **Step 1:** Attempt to unify the outer functors m and n .
 - **Result: Failure** due to a **Clash**. The functors m and n are different.
4. **unify** [$p([v \mid [V \mid VV]])$, $p([v \mid V] \mid VV)$]
 - **Step 1:** Unify the outer functors: $p = p$. Proceed to the list arguments.
 - **Step 2:** Unify $[v, V|VV]$ with $[[v|V]|VV]$. This translates to unifying the head and the tail of the lists.
 - **Step 3:** Head Unification: Attempt to unify the atom v with the list $[v|V]$.
 - **Result: Failure** due to a **Clash**. An atom (v) cannot unify with a list structure ($[v|V]$).
5. **unify**[$g([T])$, $g(T)$]
 - **Step 1:** Unify the outer functors: $g = g$. Proceed to the arguments.
 - **Step 2:** Unify $[T] = T$.
 - **Result: Failure** due to the **Occurs Check**. The variable T occurs inside the list $[T]$.

Question 3.3: Proof Tree

- a. Proof tree branches and substitutions for: $\text{?- unary_plus}([1|X], [1, 1|Y], [1, 1, 1|X])$.
- **Level 0:** $\text{unary_plus}([1|X], [1, 1|Y], [1, 1, 1|X])$
 - Unifies with Rule 2. Substitution 1: $X_1 = [1|X], A_1 = 1, Y_1 = [1|Y], Z_1 = [1, 1|X]$.
 - **Level 1:** $\text{unary_plus}([1|X], [1|Y], [1, 1|X])$
 - Unifies with Rule 2. Substitution 2: $X_2 = [1|X], A_2 = 1, Y_2 = Y, Z_2 = [1|X]$.

- **Level 2:** `unary_plus([1|X], Y, [1|X])`

From **Level 2**, the tree splits into two paths:

- **Path A (Rule 1):** `unary_plus(X3, [], X3) :- unary_number(X3).`
 - Substitution 3A: $Y = [], X_3 = [1|X]$.
 - **Level 3A:** `unary_number([1|X]).`
 - From here, `unary_number` resolves infinitely:
 - **Leaf 1 (Success):** Base case if $X = []$. **Answer:** $X = [], Y = []$.
 - **Leaf 2 (Success):** Recursive case $\rightarrow$ base case if $X = [1]$. **Answer:** $X = [1], Y = []$.
 - **Leaf 3 (Success):** Recursively if $X = [1, 1]$. **Answer:** $X = [1, 1], Y = []$.
 - **Path B (Rule 2):** `unary_plus(X4, [A4|Y4], [A4|Z4]) :- unary_plus(X4, Y4, Z4).`
 - Substitution 3B: $X_4 = [1|X], Y = [A_4|Y_4], [1|X] = [A_4|Z_4] \Rightarrow A_4 = 1, X = Z_4$.
 - **Level 3B:** `unary_plus([1|Z_4], Y_4, Z_4).`
 - Unifying with Rule 1 fails (Occurs check: trying to unify $[1|Z_4]$ with Z_4).
 - Unifying with Rule 2 leads to an **infinite failure branch** as the first argument grows indefinitely while the third shrinks.
- b.** It is a **success proof tree** because we can find at least one success leaf in it. However, the tree itself is **infinite**. This happens because `unary_number` can generate numbers forever, and the second rule of `unary_plus` creates a branch that fails infinitely.
- c.** **Yes**, it is provable. Since there is a success leaf, Prolog will be able to prove it and will return the first valid substitution it finds upon execution, which is $X = []$ and $Y = []$.
- d.** **No, L9 is undecidable.** Regular rational logic programming is decidable because it lacks functors, so we can only build a finite number of basic terms. But once we add the `cons` functor, we can nest it inside itself infinitely to create lists of any length. Because we can create infinitely large nested data structures, the language can simulate an infinite memory tape (like a Turing machine), making it Turing complete and therefore undecidable.